When Timely Answers Become Obsolete: A Model-Based Study of State-Validity-Aware Inference Scheduling

Ekansh Chawla

12 October 2026
Technical report, version 1.0.1 – Simulation and Apple GPU experiments
DOI: 10.5281/zenodo.23328001

Abstract

An inference response can meet a completion deadline yet lose relevance when the application state changes. We investigate whether historical state-change rates can improve scheduling among currently valid requests after cancellation is enabled for every competitor. A discrete-event model represents chunked prefill, continuous decode batching, bounded KV capacity, delayed cancellation and eviction/recomputation. Across 31 fixed scenarios and 20 independent confirmatory seeds, a survival-weighted diagnostic heuristic passes a predefined practical screen in 3 scenarios. In a high-change, overloaded case it improves modeled useful-decision throughput by 42.8%, but raises absolute opportunity success only from 3.54% to 5.05%. CPU mechanism controls retain a conditional gain after removing synchronized deadlines. Separate Apple GPU confirmation finds an effect in high-change synchronized conditions, while its asynchronous controls do not retain an advantage over the matched policy without the rate term. High-change agents can receive substantially worse service. These outcomes justify a narrow follow-up, not a serving-system performance claim. The timings are uncalibrated; separate Apple GPU experiments measure real generation and reveal timing-pattern sensitivity. No new algorithmic novelty or superiority over published serving systems is established.

1 Introduction

Inference scheduling influences which agent decisions are available when needed. Deadline satisfaction alone is insufficient when a relevant observation changes after a request starts. A completed response for an abandoned state may be useless even if generation finishes promptly. Conversely, a state update need not invalidate every earlier response: relevance is an application contract.

This study tests a limited question: after all schedulers can cancel invalid work, does using past relevant state-change rates improve the allocation of service to work that is still valid? The experiment deliberately gives cancellation to the baseline. Comparing an invalidation-aware policy only against an engine that continues every abandoned request would confound mechanisms.

The artifact contributes an explicit decision-opportunity metric, a replay model, controlled comparisons and negative-regime analysis. It is a technical report combining a modeled opportunity with separate, bounded real-inference evidence. Matched engine-integrated evaluation remains necessary to validate the simulated cache/preemption mechanism.

2 Related work and novelty boundary

Deadline and SLO scheduling are established areas. JITServe integrates uncertain request information and batching for heterogeneous application objectives [1]. TimelyLLM aligns segmented generation with physical execution [2]. These works invalidate the broad claim that agent-aware deadlines and batching are unexplored. Agentix addresses program-level scheduling [3]; Sarathi-Serve addresses phase interference [4].

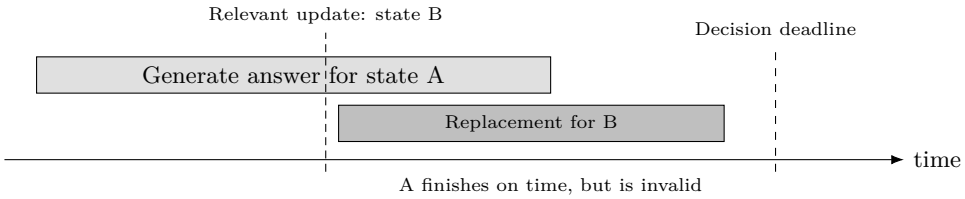


Figure 1: Illustrative validity contract, not a measured trace. The decision needs the current relevant state. Completion before a deadline does not establish usefulness.

Retire's October 2026 preprint addresses coordinated invalidation and state inheritance for revisable executions [5]. This report does not propose versioned handoff or reproduce that implementation. Its prospective priority test is a possible additional policy question, not an established research gap. Freshness-aware computation also appears in age-of-information work on preemption and waiting [6]. Our binary validity at fixed decision points differs from average age, but that difference alone is not algorithmic novelty. Belief-based scheduling for age of incorrect information also predates this project [7]; predictive freshness is not claimed as a new principle.

The accompanying literature review compares 15 core serving papers and additional boundary work. Local FIFO, EDF and SRPT-style controls must not be described as reproductions of those systems. Publication-level comparisons would require actual artifacts with matched backends, resource limits and workload contracts.

3 Problem formulation

Agent i has decision windows j, each ending at an externally specified deadline d_{ij}. A window begins with a fresh observation. Relevant updates advance the observation version v and produce a replacement request; irrelevant updates leave validity unchanged. At a decision point, at most one answer can satisfy the opportunity. No response from a different window is accepted.

Let c_r be the completion time of request r, and $v_{ij}(d)$ the latest relevant version within decision window j at its closing deadline. Version names are scoped to that window; the next window begins a separate opportunity. Define

$$Y_{ij} = \mathbf{1}\{\exists r : i_r = i,\ j_r = j,\ c_r \le d_{ij},\ v_r = v_{ij}(d_{ij})\}. \tag{1}$$

The primary modeled goodput and opportunity success rate are

$$G = \frac{\sum_{i,j} Y_{ij}}{T}, \qquad S = \frac{\sum_{i,j} Y_{ij}}{|\mathcal{O}|}. \tag{2}$$

Here $\mathcal{O}$ includes all evaluated opportunities, including those with no admitted request. The denominator cannot shrink when a scheduler rejects work. Warmup is excluded from both opportunity counts and the evaluation duration.

Real applications also require a quality predicate $Q(r) \ge q_{\min}$. The model assumes every completed response is usable, conditional on current version; it does not test that predicate with an LLM. Thus Y is modeled service success, not route correctness, robot safety or successful autonomous behavior.

3.1 Why EDF is a control, not a universal optimum

Consider four independent jobs arriving at zero on one preemptive server, with unit value for an on-time completion. Job A requires three time units and has deadline 3; B, C and D each require one unit, with deadlines 3.1, 3.2 and 3.3. EDF completes A first and leaves no remaining job able to finish on time. Serving B, C and D first completes three on time while abandoning A. This constructed example is ordinary overload scheduling, not a new theorem. It shows why a deadline-feasibility objective and a useful-throughput objective need different evaluation, even before batching and state changes enter.

3.2 A useful elementary calculation

Suppose relevant state changes are a homogeneous Poisson process of rate λ, and a response remains useful only if no change occurs before use. For a known additional interval u,

$$\Pr(\text{no relevant change in } u) = e^{-\lambda u}. \tag{3}$$

This follows from the zero-event probability of the Poisson distribution. It is an elementary identity, not a new theorem or an optimal scheduling rule. Under these assumptions, waiting does not make a state intrinsically more stable: the residual time to the next update remains exponential. Debouncing can still change resource use and coalesce requests, so its system effect must be measured.

The candidate below uses predicted remaining *service* time as a ranking surrogate. The actual decision metric requires validity until d_{ij}, not merely until completion. Consequently its exponential term is not a calibrated probability of satisfying Y_{ij}, nor a derivation of an optimal policy.

4 Execution model and policies

At every kernel boundary the scheduler can select a new batch. A prefill request processes up to 64 prompt tokens; a decoding request advances by one token. A shared iteration budget enforces $P + D \leq 256$, with at most eight selected sequences. Iteration time is modeled as

$$\tau(P, D) = 0.35 + 0.012P + 0.12D \quad \text{milliseconds.} \tag{4}$$

These coefficients are assumed scenario parameters. They were not fitted to a GPU. Context-dependent attention costs, kernel dispatch, bandwidth saturation, parallelism and real cache layouts are not represented faithfully by this law. Sensitivity controls scale it; scaling is not hardware validation.

All requests have a 128-token prompt. Two observable output-length classes have means 32 and 96 tokens, independently crossed with two state-change groups. Realized lengths are lognormal, rounded and clipped to 384 tokens. Policies know the class mean and generated-token count, never the realized future length. The base cache budget is 4,096 modeled tokens. Admission conservatively reserves prompt length plus the maximum output, rather than growing blocks dynamically.

Non-selected resident requests retain their modeled cache until admission needs capacity. Eviction clears generation progress and requires recomputation, with an optional additional restart delay. No CUDA kernel is interrupted in flight: mutation invalidates an answer immediately, but cancellation releases resources only at a supported modeled boundary after the configured notification delay. During a nonzero notification delay, invalid work remains backend-eligible until its cancellation takes effect; this sensitivity represents a stale backend view, not instantaneous version-aware execution. The main grid uses zero notification delay. Completed obsolete output cannot satisfy the decision predicate.

4.1 Competitors

FIFO orders by arrival. EDF orders by absolute deadline, breaking ties by arrival. Least laxity orders by deadline minus predicted work. Predicted SRPT orders by estimated remaining work. Debounced EDF delays first admission by eight milliseconds while maintaining the same cancellation support. An EDF *without* cancellation arm is explicitly a mechanism ablation.

The remaining-work surrogate is

$$\widehat{w}_r = 0.012 \max(0, p_r - p_r^{\text{done}}) + 0.47 \max(1, \widehat{n}_r - n_r^{\text{done}}). \tag{5}$$

It approximates serial decode service. It does not predict completion under the actual future batch mix. All policies using estimates receive the same surrogate. All policies expire incomplete requests at the end of their decision window.

4.2 Diagnostic candidate

For agent i, estimate relevant changes per millisecond from its observed count divided by elapsed trace time. No ground-truth group rate or future event is used. The survival-weighted score is

$$H_r = \frac{\exp(-\widehat{\lambda}_i \widehat{w}_r)}{\widehat{w}_r} \left(1 + \frac{t - a_r}{\Delta}\right) F_r, \tag{6}$$

where a_r is arrival, Δ the base decision period, and $F_r = 1$ if predicted work fits the remaining deadline slack, otherwise $F_r = 0.01$. Larger scores run first. Aging moderates prioritization but supplies no fairness guarantee. The constants were fixed before the initial pilot, with no tuning search.

No-hazard, no-aging and no-feasibility variants remove exactly one factor. The no-hazard variant is an especially important control: otherwise a win could come entirely from ordinary size preference, aging or feasibility weighting.

5 Experimental protocol

After correctness checks and a seed-42 smoke run, the initial protocol was locally frozen before the study pilot. The parameter grid crosses 8, 16 and 32 agents; base relevant change rates 0, 2, 8 and 24 per second; and decision periods 80 and 160 ms, for 24 grid scenarios. The second state-change group has twice the base rate. Seven sensitivity scenarios vary cancellation delay, restart cost, cache size, length heterogeneity, tail variation, irrelevant updates or engine coefficients. The first five periods are warmup; 40 periods contribute to the primary metric.

Five pilot seeds and 20 disjoint confirmatory seeds yield 1,085 and 4,340 policy runs respectively. Identical immutable state/length traces are replayed across policies. Twenty new seeds produce 1,120 exploratory runs across eight additional mechanism and deadline controls. These were designed after observing the pilot, and are explicitly not an independently preregistered confirmatory study.

For each scenario, the strongest initial baseline is selected by its confirmatory mean. This is post-selection and must be visible in reporting. Descriptive 95% intervals resample paired independent-seed differences 2,000 times, retaining within-run dependence. They are not simultaneous confidence guarantees after baseline selection or multiple-scenario testing. There is no acceptance claim based on a p-value. The simulation's reproducibility does not remove model error.

The initial practical screen requires at least a 10% relative gain over the strongest initial baseline, in two regimes, with no group losing over five percentage points of success. It is a project triage rule, not a publication criterion. All scenarios are retained; passing this screen does not overrule quality, plausibility or fairness concerns.

6 Results

6.1 A conditional opportunity, not a broad win

3 of 31 confirmatory scenarios pass the practical screen. At 32 agents, 24/s base change rate and an 80-ms period, predicted SRPT achieves 14.16 useful decisions per modeled second; survival reaches 20.22, a 42.8% relative gain. There are 400 opportunities per second. Absolute success therefore rises from 3.54% to 5.05%, only 1.52 percentage points. This is not a healthy operating point, and a large relative improvement should not hide that.

At low load, differences are tiny. At eight agents and high change rates, the survival policy loses to debounced EDF. Figure 2 shows the full grid, including negative regimes. The complete table also retains sensitivity cases.

	Decision period: 80 ms					Decision period: 160 ms			
	0	2	8	24		0	2	8	24
8 agents	-0.1%	-0.2%	-0.0%	-23.0%	8 agents	-0.1%	-0.1%	-0.1%	-20.2%
16 agents	+0.0%	+0.4%	+1.4%	+1.8%	16 agents	-1.3%	+1.0%	+1.4%	+2.1%
32 agents	+0.0%	+1.4%	+8.7%	+42.8%	32 agents	+0.0%	+0.2%	+3.7%	+31.3%

Relevant state changes per second (base-rate group)

Figure 2: Confirmatory relative throughput difference against the strongest initial baseline in each scenario. Signed values indicate gains and losses; shading carries no additional information. Each cell aggregates 20 independent paired seeds. All values are simulation results.

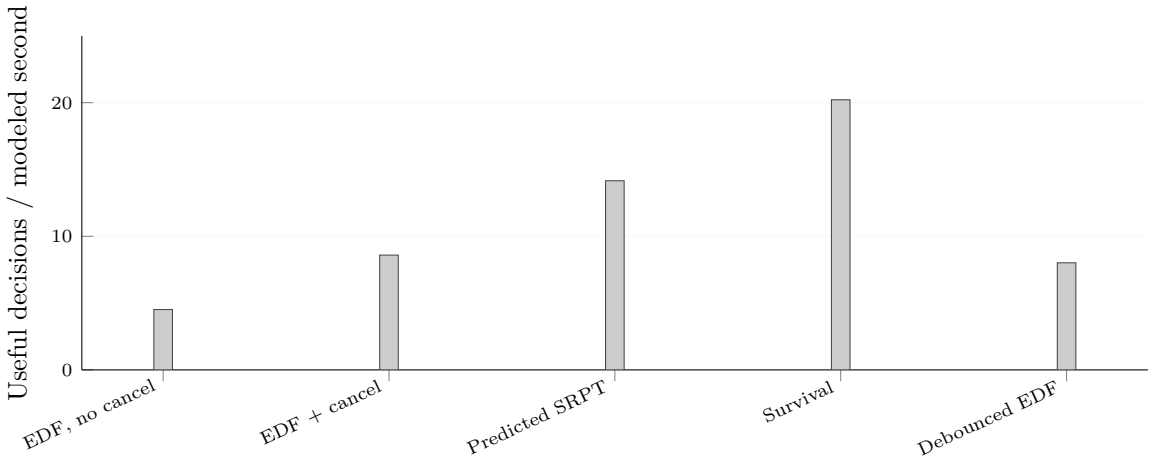


Figure 3: High-change overload example: 32 agents, 80-ms windows and base rate 24/s. Cancellation support is equal except in the labeled no-cancel arm. High relative gain coexists with low absolute success.

6.2 Cancellation must be equalized

Figure 4 separates removal of obsolete work from a priority-policy change. Cancellation alone materially changes the high-update workload. An unequal comparison would therefore overstate a scheduling contribution. In the initial synchronized workload, FIFO and EDF coincide: all surviving requests in a window have the same deadline. This makes asynchronous deadline controls necessary rather than optional.

6.3 Exploratory attribution and deadline controls

The no-hazard arm preserves size preference, aging and feasibility weighting. Its comparison with survival isolates the historical rate factor in this model. The overloaded high-change example retains a positive effect; the separate staggered and heterogeneous-period controls also retain it. In those controls, agent phases span a base period and half the agents have 1.5 times the base period, respectively. Only complete windows within the horizon are generated. Opportunity counts and denominators follow their actual deadlines.

These outcomes weaken the explanation that equal, synchronized deadlines alone cause the observed gain. They do not test non-Poisson revisions, imperfect relevance labeling, prefix inheritance or real decoder timing. All exploratory effects and intervals are reported in Appendix B.

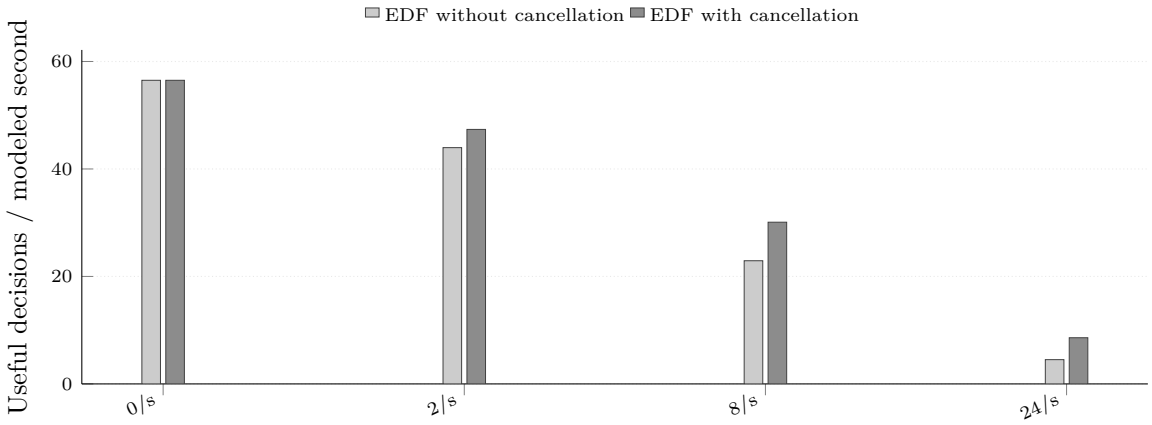


Figure 4: EDF with and without cancellation at 32 agents and an 80-ms decision period. The horizontal labels are base-group change rates; the other group has twice that rate. This figure diagnoses a mechanism, not state-of-the-art superiority.

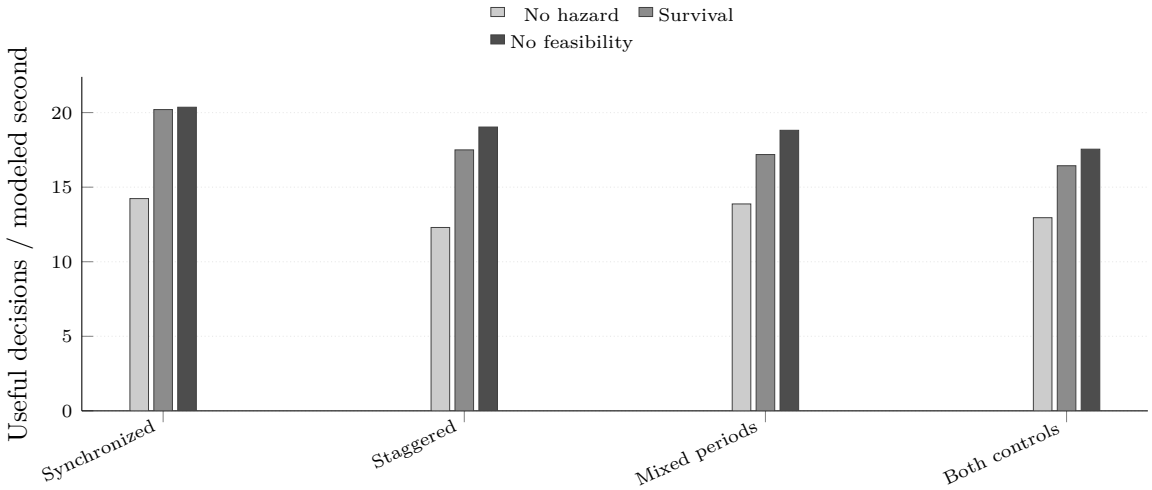


Figure 5: Exploratory mechanism controls on disjoint seeds. No-hazard removes the state-change factor; no-feasibility removes the slack weighting. The remaining components and resource model are held fixed.

6.4 Fairness defeats a simple success narrative

Figure 6 shows the high-change overload control. Adding the rate term improves the lower-change group but reduces the higher-change group's already-low success. The latter decline can fit under the original five-point absolute tolerance while representing a large proportional loss. Thus the predefined screen is too permissive to establish acceptable service allocation.

A follow-up must specify service floors or explicit group utility weights before running another experiment. Optimizing aggregate valid decisions alone naturally favors opportunities with a higher probability of surviving. That can be unacceptable even when total goodput improves. Post hoc fairness constraints cannot be used to reinterpret the original experiment as a success.

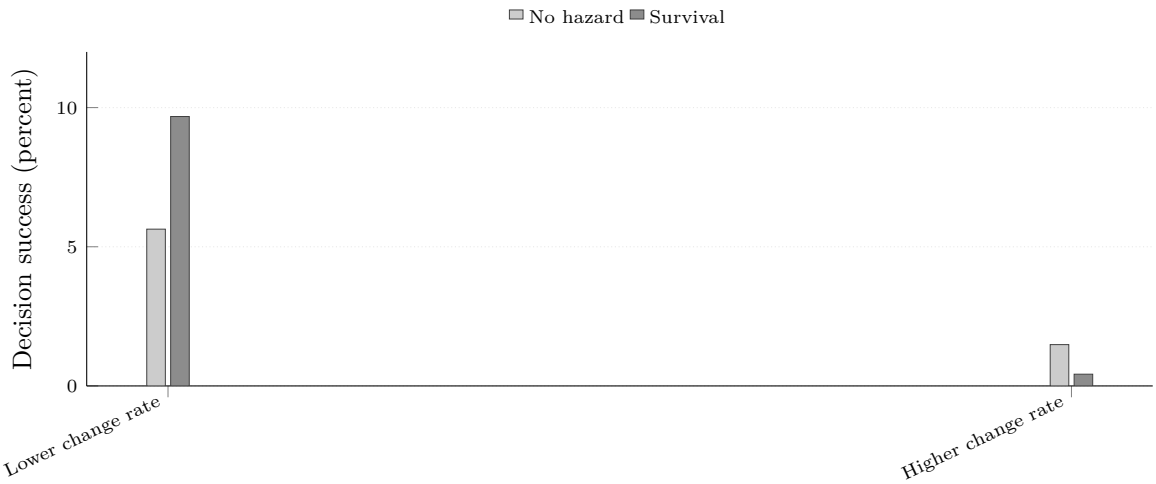


Figure 6: Exploratory group success in the high-change overload scenario. The groups differ in change rate, not in the mix of output-length classes. An absolute tolerance can mask a severe proportional disadvantage.

7 Threats to validity and next evidence gate

The modeled opportunity relies on independent Poisson changes, immediate correct relevance labels and strict latest-version validity. Some advisory plans remain useful after an update, while many real revisions are bursty or depend on an agent's actions. A closed-loop workload can change the arrivals when policies change decisions; open-loop replay alone cannot establish its application benefit.

The linear kernel law and full-length cache reservation may change policy rankings. Dynamic block allocation, prefix sharing, cache handoff and actual iteration/cancellation delays require a real engine. The historical hazard estimate lacks uncertainty intervals and assumes a stationary process. Output length predictions use class means; missed predictions can cause inappropriate priorities and eviction. Scheduler CPU wall time is not charged to modeled GPU service. The model omits transport and buffered-output publication races.

Selected-completion latency excludes failed opportunities; it is therefore a diagnostic, not a full-user-latency claim. Tokens executed after invalidation and work spent on requests that never satisfy a decision are distinct diagnostics. The artifact keeps both rather than treating every cancelled token as the same kind of waste. No p99 claim is made from short simulations.

The next evidence gate requires one NVIDIA GPU, a pinned small model and backend, actual abort-to-cessation and KV-release measurements, independent replay seeds, semantic output checks and a closest-paper artifact. The native admission experiment does not validate the modeled active-batch or cache policy. NVIDIA hardware access and rental funding are currently absent; the separate Apple GPU pilot uses existing hardware. These missing comparisons limit the scope of a systems-conference contribution.

8 Reproducibility

The companion artifact includes simulation and analysis runners using Python's standard library, plus boundary/accounting tests, per-run JSONL/CSV, trace hashes, locally frozen protocols and source/output manifests. The original simulator version is preserved in `archive/cpu-gate-1/`, so later exploratory changes do not silently alter the source used for the initial grid. Eleven CPU tests cover deterministic replay, deadline equality, late completion, stale answers, in-flight cancellation, irrelevant updates, future-length visibility, conservation and mixed deadlines. A hand-constructed two-agent case has a proved offline upper bound of one useful decision: FIFO attains it while EDF completes none. This is a diagnostic fixture, not an oracle comparison over the main benchmark grid.

To rerun the exact initial numerical study, execute `python3 archive/cpu-gate-1/src/run_study.py -phase confirmatory`. To reproduce the added controls, execute `python3 src/run_ablations.py`. Manifest wall-clock metadata varies; deterministic outcome fields should match. No external dataset download, model access token or paid service is required.

9 Separate Apple GPU evidence

The available Mac has an Apple M5 GPU and 16 GB of unified memory. An additional local study uses MLX 0.32.3 and MLX-LM 0.32.0 with a pinned 4-bit Qwen3-0.6B checkpoint [8, 9]. It is separate from the CPU simulation: there is no CUDA engine, no GPU-calibrated simulation claim, and no full implementation of the simulated preemption/cache policy. The model revision is recorded in the artifact.

9.1 Measured execution costs

Twelve homogeneous conditions cross batch sizes 1, 2, 4 and 8 with prompts of 32, 128 and 512 tokens. Each has two warmup and seven measured repetitions, with 16 subsequent decode steps. Condition order is randomized once. Each timing synchronizes device work and includes host dispatch/wait; it is not a pure GPU event duration. Synthetic repeated-token inputs isolate execution cost rather than application quality. These measurements demonstrate why the CPU timing law must not be interpreted as measured hardware performance.

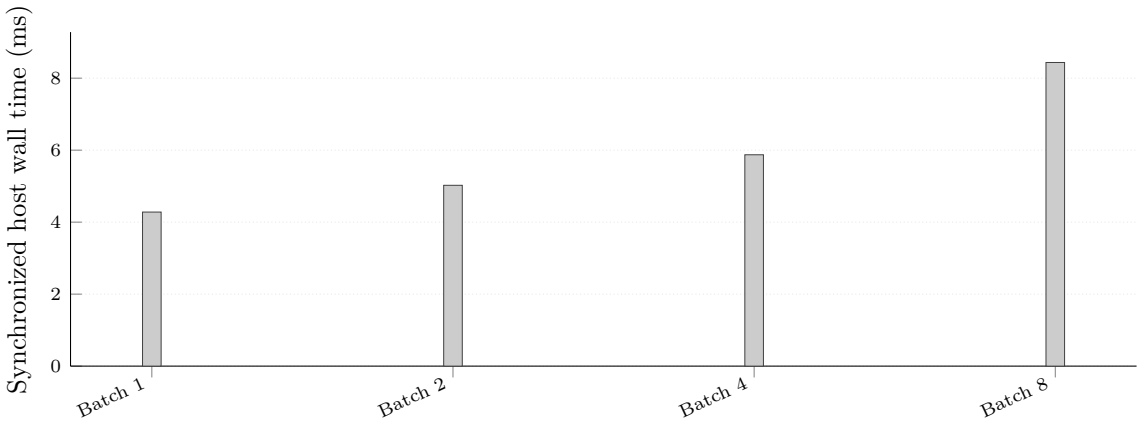


Figure 7: Measured Apple M5/Qwen3-0.6B decode costs at a 128-token prompt. Each bar is the median of seven repetition-level mean decode latencies. These are actual local execution measurements, not simulated kernel times.

9.2 Real inference with quality and cancellation

After the exploratory pilot, the unchanged admission policies were frozen for a confirmation that crosses 8/16 agents and base change rates 1/4 per second, with twenty disjoint independent seeds and four policies: 320 real replay runs. Each replay has eight 600-ms periods, excluding the first two from evaluation. Policy order is randomized within each trace. The unchanged MLX continuous batcher has an active cap of two, 64-token prefill chunks and a 160-token response limit. Every policy can remove obsolete sequences at observed control boundaries. Active requests are not reprioritized or evicted; the policy chooses admission only. The cap of two is an experimental constraint, not a demonstrated memory limit or an optimized configuration for this GPU. The policy effect may change after tuning concurrency. Native "predicted SRPT" identifiers therefore mean predicted-size *admission*, not full SRPT preemption.

The task is synthetic structured-response copying. A request asks for a version and a four- or twelve-element route payload. A response must terminate normally and parse to exactly that object; one surrounding

Markdown JSON fence is permitted. No extra fields, wrong values or truncated response are accepted. This exercises real token generation and output validation, not autonomous planning. A deterministic echo could solve the copying task without an LLM; the workload is an instrumentation test, not a business case for model use. Its 600-ms period is a predeclared experimental setting, not a production-derived agent deadline. Input tokenization is prepared before replay. Every decision opportunity is counted, including when an incorrect or incomplete response is returned.

State events that occur during a device iteration are applied before output publication. An answer is usable only if its version is still current and its observed generation availability precedes the decision. Availability is the host observation after device synchronization; subsequent payload validation is not included in that timestamp, so this is not end-to-end validated-action latency. Historical event counts, task format length priors and a common timing surrogate determine admission; realized generation length is not supplied to the policy. The larger engine experiment would need an asynchronous event/control path rather than this single host loop.

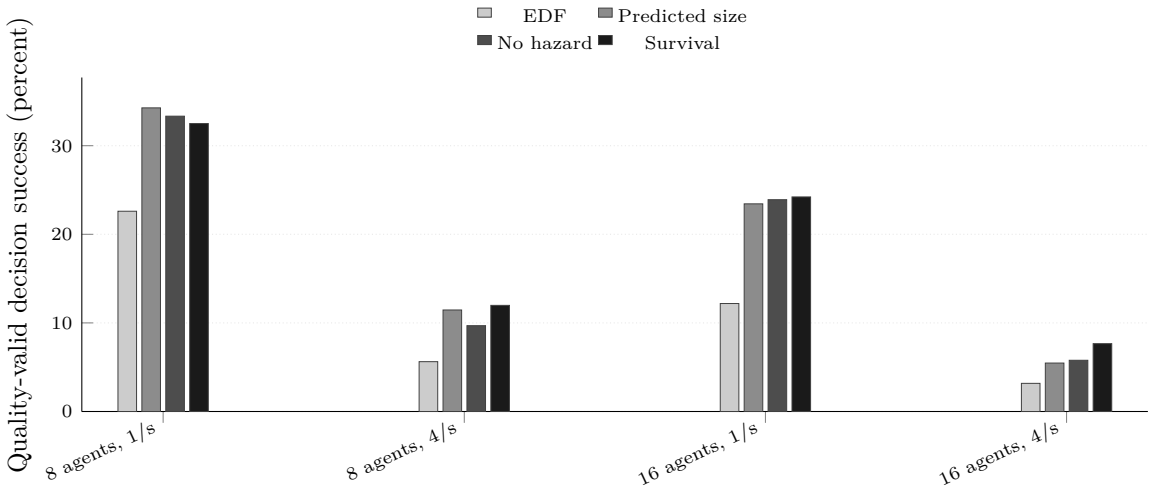


Figure 8: Actual Apple GPU admission-confirmation results. Success requires the current version, on-time availability and the fixed payload-quality check. Twenty paired seeds per condition support a bounded single-task evaluation, not a stable tail-latency or state-of-the-art claim.

Table 1: Apple GPU confirmation: opportunity success and matched hazard effect.

Agents	Rate/s	EDF	Size	No hazard	Survival	Hazard paired 95% interval
8	1	22.6	34.3	33.3	32.5	[-0.28, 0.06]
8	4	5.6	11.5	9.7	12.0	[0.15, 0.47]
16	1	12.2	23.4	23.9	24.2	[-0.24, 0.40]
16	4	3.2	5.5	5.8	7.7	[0.24, 0.78]

The four policy columns are mean opportunity success percentages. The final column is a descriptive paired interval for survival minus no-hazard throughput, in decisions per actual second. Twenty seeds and this small task cannot establish a general hardware benefit; the initial CPU screening rule is not applied to these different workloads. Both positive and negative effects are retained.

The strongest ordinary control differs by condition. The table below selects that control by its observed mean and retains the losses. Its intervals are descriptive after selection. The fixed no-hazard comparison above is the cleaner attribution test for the state-rate factor.

Table 2: Apple GPU comparison with the strongest observed control in each condition.

Agents	Rate/s	Best control	Relative gain	Paired difference interval
8	1	Size	-5.2%	[-0.46, -0.03]
8	4	Size	+4.5%	[-0.08, 0.19]
16	1	No hazard	+1.3%	[-0.24, 0.40]
16	4	No hazard	+32.4%	[0.24, 0.78]

Across recorded removal events, median signal-to-removal latency is 13.34 ms and the pooled 95th percentile is 34.46 ms. These include delay until the host regains control plus engine removal and synchronization. Pooled events are correlated and not an independent-run confidence interval. Logical cache byte counts are logged; they do not establish physical Metal allocator reclamation.

Quality pass rates among completed requests also differ with the selected task mix. The table below pools completions only as a diagnostic. It must not be read as improved underlying model accuracy, and it does not replace the all-opportunity success denominator.

Table 3: Payload quality among completed native requests (conditional diagnostic).

Policy	Completed	Payload passes	Conditional pass rate
EDF	2117	1455	68.7%
Predicted size	2506	2389	95.3%
No hazard	2528	2404	95.1%
Survival	2665	2427	91.1%

9.3 The group trade-off persists on real inference

The recorded group outcomes reveal a similar trade-off to the CPU study. In the 16-agent high-change confirmation, the higher-change group's success falls from 1.56% to 0.21% when the hazard term is enabled, even as aggregate throughput improves. This is a large proportional loss that an absolute five-point screen would miss. The methods have no explicit service floor or application-urgency weights, so the aggregate improvement is not a deployment recommendation.



Figure 9: Measured group trade-off at 16 agents and base rate 4/s. The higher-change group sees rate 8/s. Quality and deadline conditions are identical for both policies; these values concern useful decisions, not GPU utilization.

Table 4: Apple GPU confirmation: within-group opportunity success (percent).

		Lower change rate		Higher change rate	
Agents	Rate/s	No hazard	Survival	No hazard	Survival
8	1	37.08	38.54	29.58	26.46
8	4	16.25	21.67	3.12	2.29
16	1	30.73	32.92	17.08	15.52
16	4	10.00	15.10	1.56	0.21

Values are mean within-group opportunity success percentages. The task-length classes are crossed with change-rate groups, rather than assigning all long answers to one group.

The hardware experiment establishes that the replay, validation and cancellation path can run a real local model. It does not reproduce the CPU candidate's eviction mechanism, implement Retire-style handoff, or compare with published serving artifacts. Longer traces, independent run days, application-grounded deadlines, service floors and an engine-integrated matched-baseline evaluation remain necessary before a deployment or systems-conference claim.

9.4 Asynchronous controls restrict the conclusion

Supplemental hardware controls are retained in the appendix. The state-rate term does not improve the matched no-hazard policy in the tested staggered-deadline cases. Thus the positive synchronized results do not generalize across the tested timing patterns. Low-load zero-change runs meet every opportunity under all policies; repeated zero-change inputs are measurement repetitions, not independent arrival seeds.

10 Conclusion

The CPU model exposes a conditional benefit from historical state-change rates. Actual local inference confirms an effect in synchronized high-change conditions, but not a clear benefit at lower rates or in the tested asynchronous controls. Higher aggregate goodput can sharply reduce success for high-change agents. The result supports a reproducible technical report and a narrower, fairness-constrained investigation. It does not establish a novel optimal policy or superiority over published inference-serving systems. Application-grounded workloads, matched engine-integrated baselines and service guarantees are the next necessary tests before a systems-conference claim.

References

[1] W. Zhang et al. JITServe: SLO-aware LLM Serving with Imprecise Request Information. NSDI, 2026. https://www.usenix.org/conference/nsdi26/presentation/zhang-wei.

[2] N. Ling, G. Chen, A. Khandelwal and L. Zhong. TimelyLLM: Time-sensitive LLM Serving System for Physical-I/O Limited Agents. MobiSys, 2026. https://doi.org/10.1145/3745756.3809203.

[3] M. Luo et al. Agentix: An Efficient Serving Engine for LLM Agents as General Programs. NSDI, 2026. https://www.usenix.org/conference/nsdi26/presentation/luo.

[4] A. Agrawal et al. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. OSDI, 2024. https://www.usenix.org/conference/osdi24/presentation/agrawal.

[5] Y. Zhang et al. Serving a Revisable World: Versioned Execution for Interruptible Agents. arXiv:2610.01160, 1 October 2026. Preprint. https://arxiv.org/abs/2610.01160.

[6] A. Arafa, R. D. Yates and H. V. Poor. Timely Cloud Computing: Preemption and Waiting. Allerton, 2019. https://webpages.charlotte.edu/aarafa/allerton19.pdf.

[7] S. Kriouile and M. Assaad. Minimizing the Age of Incorrect Information for Real-time Tracking of Markov Remote Sources. arXiv:2102.03245, 2021. Preprint record. https://arxiv.org/abs/2102.03245.

[8] MLX-LM developers. MLX-LM: Run LLMs with MLX. Software version 0.32.0. https://github.com/ml-explore/mlx-lm.

[9] MLX Community. Qwen3-0.6B-4bit. Model checkpoint, accessed 12 October 2026. https://huggingface.co/mlx-community/Qwen3-0.6B-4bit.

A Complete initial scenario results

Table values are confirmatory means over 20 paired seeds. Throughput and paired intervals are useful decisions per modeled second. Intervals are descriptive after selecting the strongest baseline; they are not simultaneous inference.

Table 5: Complete initial simulation results: confirmatory means and descriptive paired intervals.

Scenario	Strongest control	Control	Survival	Gain	Paired 95% interval
grid_n8_r0_p80	laxity	60.95	60.92	-0.1%	[-0.28, 0.20]
grid_n8_r0_p160	laxity	49.03	48.99	-0.1%	[-0.07, -0.02]
grid_n8_r2_p80	predicted_srpt	50.72	50.64	-0.2%	[-0.16, 0.00]
grid_n8_r2_p160	laxity	43.02	42.96	-0.1%	[-0.11, 0.00]
grid_n8_r8_p80	fifo	31.41	31.39	-0.0%	[-0.22, 0.20]
grid_n8_r8_p160	predicted_srpt	24.67	24.65	-0.1%	[-0.09, 0.03]
grid_n8_r24_p80	debounce_edf	9.91	7.62	-23.0%	[-2.66, -1.88]
grid_n8_r24_p160	debounce_edf	6.14	4.90	-20.2%	[-1.42, -1.05]
grid_n16_r0_p80	predicted_srpt	82.03	82.03	+0.0%	[0.00, 0.00]
grid_n16_r0_p160	fifo	71.25	70.30	-1.3%	[-1.38, -0.50]
grid_n16_r2_p80	predicted_srpt	69.94	70.23	+0.4%	[-0.02, 0.59]
grid_n16_r2_p160	predicted_srpt	54.06	54.59	+1.0%	[0.23, 0.80]
grid_n16_r8_p80	predicted_srpt	43.42	44.02	+1.4%	[0.31, 0.89]
grid_n16_r8_p160	predicted_srpt	29.14	29.55	+1.4%	[0.17, 0.66]
grid_n16_r24_p80	predicted_srpt	11.55	11.75	+1.8%	[0.00, 0.42]
grid_n16_r24_p160	predicted_srpt	7.17	7.32	+2.1%	[0.07, 0.23]
grid_n32_r0_p80	predicted_srpt	95.56	95.56	+0.0%	[0.00, 0.00]
grid_n32_r0_p160	predicted_srpt	99.14	99.14	+0.0%	[0.00, 0.00]
grid_n32_r2_p80	predicted_srpt	80.33	81.47	+1.4%	[0.06, 2.19]
grid_n32_r2_p160	predicted_srpt	81.49	81.62	+0.2%	[-0.06, 0.31]
grid_n32_r8_p80	predicted_srpt	50.30	54.69	+8.7%	[3.22, 5.58]
grid_n32_r8_p160	predicted_srpt	43.99	45.63	+3.7%	[1.30, 2.05]
grid_n32_r24_p80	predicted_srpt	14.16	20.22	+42.8%	[5.47, 6.62]
grid_n32_r24_p160	predicted_srpt	9.52	12.49	+31.3%	[2.61, 3.38]
cancellation_delay	predicted_srpt	40.88	41.41	+1.3%	[0.19, 0.88]
restart_penalty	predicted_srpt	43.42	43.88	+1.0%	[0.20, 0.73]
small_cache	predicted_srpt	54.52	55.73	+2.2%	[0.75, 1.70]
uniform_lengths	fifo	16.11	19.08	+18.4%	[2.00, 3.91]
heavy_tail	predicted_srpt	56.17	57.17	+1.8%	[0.55, 1.52]
irrelevant_updates	predicted_srpt	59.25	59.61	+0.6%	[0.03, 0.69]
fast_engine	predicted_srpt	84.25	85.89	+1.9%	[1.12, 2.16]

The grid identifiers encode agents (n), base changes/s (r) and base period/ms (p). Sensitivity cases use 16 agents, 8 changes/s and 80-ms periods. The cache control uses 2,048 tokens; restart adds 5 ms; cancellation notification adds 4 ms; the heavy-tail control uses lognormal $\sigma = 1$; half of updates are irrelevant in that control; fast-engine coefficients are all halved. Uniform-length classes both have mean 64, while per-request realized length remains uncertain.

B Exploratory hazard attribution

These comparisons use seeds 2000–2019, independent of the initial phases. Relative gain compares survival with the identical heuristic after removing the hazard factor. Paired differences and intervals are useful decisions per modeled second. They diagnose this model and do not validate an actual serving engine.

Table 6: Exploratory simulation: matched hazard effects across mechanism controls.

Scenario	Relative gain	Paired difference	Descriptive 95% interval
overload_high_change	+41.9%	5.97	[5.02, 6.89]
overload_moderate_change	+7.1%	3.58	[2.66, 4.45]
uniform_lengths	+19.9%	3.19	[2.38, 3.95]
staggered_deadlines	+42.3%	5.20	[4.58, 5.86]
heterogeneous_periods	+23.9%	3.31	[2.94, 3.70]
both_deadline_controls	+26.9%	3.48	[3.06, 3.92]
slow_cancellation	+51.1%	6.41	[5.64, 7.14]
larger_cache	+41.8%	5.95	[5.05, 6.84]

C Event semantics and interpretation

State events are processed at their trace times. A launched kernel completes before its physical resources can be cancelled or expired. A relevant update at the completion boundary changes validity; a completion exactly on its deadline counts if it supplies the requested current version. Decision-window transitions do not allow a response for the previous opportunity to satisfy the new one. The unit tests exercise these boundary cases.

Each completed kernel is charged in full, including work that becomes obsolete during execution. A kernel still in flight at the final horizon is omitted from token and busy-time diagnostics; no pending completion can satisfy that cutoff. Evicted progress is recomputed, not silently resumed with an unavailable cache. Realized output lengths are used only to decide engine completion; scheduling sees the fixed class prior and observed generation progress. These contracts prevent free mid-kernel aborts and clairvoyant length scheduling, but still do not replace GPU profiling.

D Complete Apple GPU microbenchmark

All times are median synchronized host-wall milliseconds. Decode throughput is batch tokens per second, computed from the median repetition-level mean step time. The checkpoint is quantized; these values do not transfer to an NVIDIA GPU, another model, or variable-length continuously changing batches.

Table 7: Complete Apple M5 execution microbenchmark.

Batch	Prompt length	Prefill ms	Decode step ms	Decode tokens/s
1	32	15.79	4.17	239.7
1	128	31.24	4.28	233.6
1	512	56.92	4.73	211.3
2	32	26.92	4.90	407.8
2	128	58.49	5.02	398.1
2	512	117.60	6.16	324.6
4	32	33.29	5.62	711.2
4	128	61.04	5.87	681.2
4	512	214.34	7.21	554.9
8	32	52.59	7.52	1063.9
8	128	116.72	8.44	948.4
8	512	429.73	13.62	587.5

The strict-JSON execution smoke test generated the correct object surrounded by a Markdown fence and failed its strict format check. The subsequent pilot's documented validator explicitly permits one complete surrounding fence; this contract was written before native policy results. It does not accept arbitrary prose or extract an object from an otherwise invalid answer.

The optional environment lockfile, exact model revision, completed generated outputs, per-iteration wall times, removal events and package/device manifests are supplied in `local_hardware/` and `results/apple_gpu_*`. Model weights are downloaded separately and are not redistributed in the release package.

E Initial Apple GPU pilot

The initial pilot used five independent seeds per condition, six 600-ms periods and one warmup period. It preceded the twenty-seed campaign. The policies were not tuned between phases. The table retains all pilot conditions; values are opportunity success percentages.

Table 8: Initial five-seed Apple GPU pilot: opportunity success (percent).

Agents	Rate/s	EDF	Size	No hazard	Survival
8	1	20.0	36.0	35.5	36.5
8	4	3.5	11.0	8.0	10.0
16	1	10.0	24.0	21.5	25.2
16	4	2.8	4.5	6.0	7.2

F Supplemental real-inference controls

Eighty supplemental runs cover zero relevant changes at two and sixteen agents, plus staggered deadlines at sixteen agents with base rate 4/s. The mixed-period case also gives half the agents a 1.5-times longer period. Each condition has five physical repetitions; the zero-change inputs are identical across seeds, so those are not five independent workload traces. The asynchronous conditions use five independent state-event traces. Only complete decision windows within the horizon are generated. Values below are opportunity success percentages.

Table 9: Supplemental Apple GPU controls: opportunity success (percent).

Condition	EDF	Size	No hazard	Survival
Underload, no changes	100.0	100.0	100.0	100.0
Overload, no changes	24.6	49.6	50.0	48.3
Staggered deadlines	0.0	0.0	7.3	6.0
Staggered + mixed periods	2.0	5.5	5.8	5.5

The priority term provides no intrinsic benefit when no relevant updates occur. Minor differences between identical no-hazard and zero-hazard logic arise from wall-clock execution variation under overload. More importantly, the asynchronous controls do not retain an advantage over the matched no-hazard policy. Very poor EDF/size outcomes there reflect their lack of the matched heuristic's feasibility weighting; they cannot establish a novel freshness contribution. These controls restrict the claim to the specific synchronized high-change conditions.

G Output-schema audit

After the initial native campaigns, numeric-type equality was tightened to prevent Python's boolean/integer equality from accepting a boolean as an integer version. All 11,579 retained completed outputs from the pilot and confirmation were checked again; no quality status changed. Original campaign sources remain archived by their recorded hashes. The released validator has six dependency-free tests, including stale values, truncation, extra content and incorrect numeric types.